

registro_ia.md

Registro de Interacciones con Herramientas de Inteligencia Artificial

Integrador 2 — Matemática y Programación en Python

Franco Kaddour

Introducción

Durante el desarrollo de este Trabajo Integrador II se utilizaron tres herramientas de Inteligencia Artificial con distintos propósitos: brainstorming inicial, verificación matemática y desarrollo del programa en Python. A continuación se documentan las interacciones más relevantes con cada herramienta.

HERRAMIENTA 1: ChatGPT (OpenAI)

Uso principal: Generación de ideas y comprensión conceptual de la Teoría de Conjuntos

Iteración 1.1 — Selección del tema

Prompt enviado:

“Necesito elegir un tema para un trabajo de Teoría de Conjuntos con diagrama de Venn de tres conjuntos. Tiene que ser una situación cotidiana real, no un ejemplo del libro. ¿Podés darme algunas ideas originales?”

Respuesta de ChatGPT (resumen): ChatGPT propuso varias opciones: análisis de hábitos de consumo en una cafetería, uso de plataformas de streaming, actividades en un gimnasio, y uso de redes sociales. Destacó que la opción del gimnasio era interesante porque permite intersecciones naturales (alguien puede ir a musculación, clases grupales y nadar), los datos son realistas y el universo es fácil de delimitar.

Decisión tomada: Se eligió el gimnasio OnFit con tres conjuntos: musculación (A), clases grupales (B) y entrenamiento funcional (C), con un universo de 120 socios activos.

Iteración 1.2 — Comprensión de las regiones del diagrama de Venn

Prompt enviado:

"¿Cómo se calculan las 7 regiones internas de un diagrama de Venn con tres conjuntos? Necesito entender la lógica, no solo la fórmula."

Respuesta de ChatGPT (resumen): Explicó que el diagrama de Venn con tres conjuntos tiene 7 regiones internas más la región exterior. La clave es que cuando calculamos $|A \cap B|$, ese número ya incluye a los que están en los tres conjuntos simultáneamente. Por eso, para obtener la intersección exclusiva de solo A y B (sin C), hay que restar $|A \cap B \cap C|$. Lo mismo aplica para cada conjunto individual: al calcular "Solo A", hay que restar no solo los que comparte con B, sino también los que comparte con C, y evitar contar dos veces a los del centro.

Utilidad: Esta explicación conceptual ayudó a comprender el fundamento de las fórmulas antes de aplicarlas, lo cual fue clave para implementarlas correctamente en Python.

Iteración 1.3 — Definición de datos coherentes

Prompt enviado:

"Quiero usar estos datos para mi diagrama de Venn: $|U|=120$, $|A|=65$, $|B|=55$, $|C|=40$, $|A \cap B|=22$, $|A \cap C|=18$, $|B \cap C|=15$, $|A \cap B \cap C|=6$ ¿Son datos coherentes o van a dar algún valor negativo?"

Respuesta de ChatGPT: Realizó los cálculos preliminares y confirmó que los datos son coherentes:

- Solo A = 31 (positivo ✓)
- Solo B = 24 (positivo ✓)
- Solo C = 13 (positivo ✓)
- La unión total = 111, que es menor a 120 (positivo ✓)
- Fuera = 9 (positivo ✓)

Utilidad: Permitió validar los datos del problema antes de desarrollar la resolución completa, evitando tener que replantear el ejercicio más adelante.

HERRAMIENTA 2: Google Gemini

Uso principal: Verificación matemática y aplicación del Principio de Inclusión-Exclusión

Iteración 2.1 — Verificación con el Principio de Inclusión-Exclusión

Prompt enviado:

"¿Podés verificarme este cálculo usando el Principio de Inclusión-Exclusión? $|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |A \cap C| - |B \cap C| + |A \cap B \cap C|$ Con: $|A|=65$, $|B|=55$, $|C|=40$, $|A \cap B|=22$, $|A \cap C|=18$, $|B \cap C|=15$, $|A \cap B \cap C|=6$ "

Respuesta de Gemini: Realizó el cálculo paso a paso:

$$\begin{aligned}|A \cup B \cup C| &= 65 + 55 + 40 - 22 - 18 - 15 + 6 \\ &= 160 - 55 + 6 \\ &= 111\end{aligned}$$

Confirmó que el resultado es correcto. También señaló que el Principio de Inclusión-Exclusión garantiza que no se cuente dos veces ningún elemento que pertenezca a más de un conjunto.

Utilidad: Permitió verificar independientemente que la suma de las 7 regiones internas (111) coincide con el resultado del Principio de Inclusión-Exclusión. Esto confirmó la corrección del ejercicio antes de volcarlo al documento final.

Iteración 2.2 — Interpretación de los resultados

Prompt enviado:

"Obtuve estos resultados para el diagrama de Venn de el gimnasio OnFit con 120 socios: Solo A=31, Solo B=24, Solo C=13, Solo $A \cap B$ =16, Solo $A \cap C$ =12, Solo $B \cap C$ =9, $A \cap B \cap C$ =6, Fuera=9 ¿Podés ayudarme a redactar una conclusión que interprete estos resultados de forma matemática y práctica?"

Respuesta de Gemini (resumen): Propuso una conclusión que destacaba:

- El 92,5% de los socios utiliza al menos una actividad (111/120).
- La musculación es la actividad con mayor participación exclusiva (31 socios).
- Solo el 5% aprovecha las tres actividades simultáneamente (6/120).
- La combinación más frecuente entre dos actividades es musculación + clases grupales (16 socios).
- Los 9 socios externos representan una oportunidad de mejora para el gimnasio.

Utilidad: La ayuda de Gemini enriqueció la interpretación de los resultados, agregando perspectiva práctica que va más allá del cálculo matemático.

Iteración 2.3 — Comparación entre herramientas de IA

Prompt enviado:

"¿En qué se diferencia tu manera de explicar la Teoría de Conjuntos respecto a ChatGPT?"

Respuesta de Gemini (resumen): Gemini explicó que tiende a estructurar las explicaciones de manera más visual y a verificar paso a paso, mientras que ChatGPT suele ser más conversacional y contextual. Para matemáticas formales, Gemini resulta más preciso en la verificación de fórmulas. Para brainstorming creativo, ChatGPT es más fluido.

Utilidad: Esta reflexión ayudó a entender las fortalezas de cada herramienta y usarlas estratégicamente durante el trabajo.

HERRAMIENTA 3: Claude (Anthropic)

Uso principal: Desarrollo del programa en Python y generación de todos los entregables

Iteración 3.1 — Estructura del programa Python

Prompt enviado:

“Necesito desarrollar un programa en Python que resuelva el diagrama de Venn con tres conjuntos. El programa debe pedir todos los datos al usuario, calcular las regiones, mostrar los resultados y generar el diagrama gráfico. ¿Qué estructura le recomendás?”

Respuesta de Claude (resumen): Propuso organizar el programa en funciones bien separadas siguiendo el principio de responsabilidad única:

- `mostrar_bienvenida()`: encabezado del programa
- `ingresar_entero()`: validación de entrada numérica
- `ingresar_datos()`: guía al usuario en el ingreso
- `calcular_regiones()`: aplica las fórmulas matemáticas
- `validar_resultados()`: detecta datos inconsistentes
- `mostrar_resultados()`: tabla de resultados
- `graficar_venn()`: diagrama visual con matplotlib
- `main()`: función coordinadora

Señaló que este diseño hace el código más legible, más fácil de corregir y mejor documentado con docstrings.

Utilidad: La estructura propuesta permitió organizar el desarrollo del programa de manera profesional antes de escribir la primera línea de código.

Iteración 3.2 — Implementación del diagrama de Venn en matplotlib

Prompt enviado:

"¿Cómo puedo dibujar el diagrama de Venn con matplotlib sin usar librerías externas como matplotlib-venn? Necesito que los tres círculos se superpongan correctamente y que se pueda escribir un número en cada una de las 7 regiones."

Respuesta de Claude (resumen): Explicó que con `matplotlib.patches.Circle` se pueden dibujar círculos posicionados en coordenadas específicas. Para los tres conjuntos, recomendó ubicar los centros en un triángulo equilátero:

- Círculo A: (4.1, 5.5)
- Círculo B: (6.9, 5.5)
- Círculo C: (5.5, 3.3) Con $\text{radio} = 2.3$ para que las superposiciones sean visibles y proporcionales.

Para los números en cada región, indicó calcular las coordenadas de cada zona del diagrama en base a los centros de los círculos y posicionar los textos con `ax.text()`.

Utilidad: Esta orientación técnica fue fundamental para implementar el diagrama de Venn sin depender de librerías adicionales, haciendo el programa más portable.

Iteración 3.3 — Validación del código

Prompt enviado:

"Revisá este fragmento del programa donde calculo las regiones. ¿Hay algún error matemático o de lógica? $\text{solo_a} = a - (ab - abc) - (ac - abc) - abc$ "

Respuesta de Claude: Confirmó que la fórmula es correcta y la explicó analíticamente:

- $(ab - abc)$ = socios en $A \cap B$ que NO están en $C \rightarrow$ se restan de A
- $(ac - abc)$ = socios en $A \cap C$ que NO están en $B \rightarrow$ se restan de A
- abc = socios en los tres conjuntos $\rightarrow$ se resta de A
- Lo que queda es A puro: solo musculación, sin ninguna otra actividad

También advirtió que era necesario validar que los resultados no sean negativos, ya que datos inconsistentes del usuario podrían generar valores inválidos. Sugirió implementar la función `validar_resultados()` por esa razón.

Utilidad: La verificación de la lógica matemática en el código evitó posibles errores conceptuales y mejoró la robustez del programa.

Iteración 3.4 — Revisión del código completo

Prompt enviado:

“El programa ya está terminado. ¿Podés revisarlo y sugerirme mejoras en claridad, comentarios y nombres de variables?”

Respuesta de Claude (resumen): Sugirió:

- 1. Agregar docstrings con descripción de parámetros y retorno en cada función.
- 2. Usar f-strings para los mensajes de resultado (más legibles que concatenación).
- 3. Reemplazar números “mágicos” de posición en el gráfico con variables con nombre descriptivo.
- 4. Agregar verificación final: mostrar si el total coincide con |U|.
- 5. Separar visualmente las secciones del código con comentarios de línea.

Utilidad: Las sugerencias de revisión elevaron la calidad del código de funcional a profesional, cumpliendo los estándares de código comentado y variables claras que exige el trabajo integrador.

Comparativa de las Herramientas Utilizadas

Aspecto	ChatGPT	Gemini	Claude
Brainstorming	Excelente	Bueno	Muy bueno
Explicación conceptual	Muy fluida	Estructurada	Muy detallada
Verificación matemática	Buena	Excelente	Muy buena
Desarrollo de código	Bueno	Regular	Excelente
Revisión de código	Buena	Buena	Excelente
Generación de documentos	Buena	Buena	Excelente

Ventajas observadas por herramienta:

ChatGPT:

- Mejor para generar ideas creativas rápidamente.
- Explicaciones conversacionales y accesibles.
- Responde bien a preguntas conceptuales abiertas.

Gemini:

- Más preciso en verificaciones matemáticas paso a paso.
- Bueno para estructurar información tabular.
- Integración con Google Workspace facilita la exportación de contenido.

Claude:

- Mejor para desarrollo y revisión de código Python.
 - Genera código bien comentado y con buenas prácticas.
 - Excelente para redacción técnica y generación de documentos completos.
 - Capacidad para mantener contexto a lo largo de conversaciones largas.
-

Reflexión Final sobre el Uso de IA

El uso de múltiples herramientas de IA durante el desarrollo de este trabajo permitió comprobar que no existe una única herramienta “mejor”, sino que cada una tiene fortalezas específicas. La estrategia más efectiva fue usar **ChatGPT para pensar**, **Gemini para verificar** y **Claude para construir**.

Es importante destacar que las herramientas de IA no reemplazan el proceso de comprensión: fue necesario entender los conceptos de Teoría de Conjuntos para poder formular buenas preguntas, evaluar las respuestas y detectar cuándo una sugerencia era incorrecta. La IA es una herramienta de apoyo poderosa, pero el pensamiento crítico del estudiante sigue siendo indispensable.

Por otro lado, el uso de IA redujo significativamente el tiempo dedicado a tareas repetitivas (formateo, estructura del código, redacción) y permitió enfocar la energía en la comprensión conceptual y la resolución matemática, que son los aspectos centrales del aprendizaje.

Franco Kaddour *Herramientas utilizadas: ChatGPT (OpenAI), Google Gemini, Claude (Anthropic)*